

강지연 협업일지 요약

중급프로젝트 RFP RAG · 기록 기간 2026-07-13 ~ 2026-08-03

역할 요약

강지연님은 프로젝트 PM으로 일정·역할·아키텍처와 담당 영역 간 의존성을 조율하면서, 검색·평가 파트를 함께 맡았다. 초반에는 EDA와 RAG 설계 방향을 정리하고 인원 변동에 맞춰 역할을 재배치했으며, 후반에는 Retriever, Hybrid Search, Reranker, Golden Set과 검색 성능 평가를 수행했다. 데이터 파이프라인 담당자가 만든 전처리·칭킹·Vector DB 결과를 실제 검색 관점에서 검토해 표 검색 누락, 짧은 청크 편향, 오버랩 누락과 내부 참조 ID 혼입 같은 문제를 팀에 피드백했다. 마지막에는 비교 실험으로 1024 칭킹 기반의 최종 검색 구성을 선택하고 UI·README·보고서·발표와 제출까지 연결했다.

핵심 기여

1. PM 역할과 팀 운영

- 프로젝트 Kick-off 이후 팀원이 같은 진행 상황을 볼 수 있도록 Obsidian을 협업 기록과 자료 공유 공간으로 사용하는 방향을 정리했다.
- 프로젝트 초기에 역할을 배정하고, 팀원 중도하차와 건강 문제로 인원이 바뀔 때마다 검색·Generation·데이터 파이프라인의 업무 범위를 다시 조정했다.
- 팀원들과 역할, 시스템 아키텍처와 남은 2주 일정을 함께 검토하고 우선순위를 조율했다.
- 강사 아키텍처 리뷰 결과를 팀에 반영하고, Naive RAG와 Advanced RAG를 분리해 기준선과 개선 실험을 동시에 관리하도록 진행 방향을 정리했다.
- 전처리 → 칭킹 → 임베딩·Vector DB → Retriever → Generation → 평가 순서에서 앞 단계 산출물이 다음 담당자에게 필요한 형식으로 전달되는지 확인했다.
- 표·오버랩·metadata·공용 Vector DB처럼 여러 담당 영역에 걸친 문제를 발견하면 담당자에게 공유하고 수정 결과를 다시 검색과 UI에서 확인했다.
- 최종 단계에서는 README·보고서·발표자료를 정리하고, GitHub 저장소 주소와 보고서를 LMS에 제출해 팀 산출물을 마무리했다.

PM 역할은 팀원의 작업을 대신 수행하는 것이 아니라, 각 파트가 같은 아키텍처와 일정 안에서 연결되도록 결정사항·의존성·문제·다음 행동을 지속적으로 맞추는 역할이었다.

2. 초기 EDA와 RAG 아키텍처 방향 정리

- 프로젝트 안내서를 검토하고 원본 문서의 문자 수와 문서 내용을 중심으로 초기 EDA를 수행했다.
- 분석 결과를 팀이 재사용하기 쉽도록 노트북보다 Python 스크립트 중심으로 관리하는 방향을 제안했다.
- HWP를 PDF로 변환했을 때 표 구조가 깨지거나 변환이 실패하는 사례를 확인하고, HWP 표를 직접 추출하는 방향을 조사했다.
- 원본 데이터에서 HWP 추출 실패, 중복 PDF, CSV 메타데이터 누락을 찾아 팀에 공유했다.
- 나라장터 자료를 이용해 누락 메타데이터를 보완하고, 확인할 수 없는 값은 임의로 채우지 않는 방식으로 관리했다.
- 인원 변동에 맞춰 역할을 다시 나누고, Naive RAG와 Advanced RAG를 분리해 진행하는 구조를 팀 논의에 반영했다.

3. Retriever·Hybrid Search·Reranker 구현과 연동

- Naive Vector DB를 대상으로 similarity search가 가능한 Retriever 코드를 작성하고 실제 검색 결과를 확인했다.
- Chroma에 저장된 `chunk_id`, `table_id`, 출처 파일명과 신뢰도 정보를 검색 결과에 노출하도록 생성 파트와의 연결 조건을 정리했다.
- Dense 검색과 BM25를 결합한 Hybrid Search를 구현하고, Reranker를 연결해 후보 문서를 다시 정렬하는 흐름을 구성했다.
- MMR 기반 Dense 검색과 BM25 전처리 조건을 맞추며 검색 결과가 특정 유형의 짧은 청크에 치우치지 않는지 점검했다.
- Retriever의 metadata 계약이 바뀔 때 생성 코드도 함께 수정될 수 있도록 담당자들과 인터페이스를 조율했다.

4. 전처리·청킹·Vector DB 품질 검토

검색 코드를 작성하는 데서 끝내지 않고, 실제 Vector DB와 청크를 열어 다음 문제를 확인했다.

발견한 문제	확인·제안 내용	팀 반영 방향
HWP는 실제 페이지 번호 제공이 어려움	PDF는 페이지, HWP는 문단·구조 위치로 추적해야 함	파일 형식별 위치 metadata 분리
[문서], [위치], [유형] prefix	본문 의미보다 공통 prefix가 임베딩 유사도에 영향을 줄 수 있음	검색 본문에서 제외하고 metadata로 유지
표와 일반 텍스트 혼합	KSS가 표의 번호·구조를 문장으로 잘못 해석할 수 있음	일반 텍스트만 KSS, 표는 Markdown 별도 색인
표 구조 붕괴	PDF/HWP 표의 Markdown·HTML과 실제 원본을 비교해야 함	표 샘플을 육안 검토하고 복구 규칙 개선
짧은 텍스트 청크 편향	짧은 청크가 검색 상위를 점유해 표 결과를 밀어냄	문장을 목표 토큰까지 묶는 패킹 방식 검토
오버랩 누락	설정값은 있어도 실제 연속 청크 본문이 겹치지 않는 사례 발견	원문 대조 기반 오버랩 검증 요청
내부 참조 ID 혼입	표·이미지 연결용 ID가 Dense·BM25에서 일반 단어처럼 처리됨	검색 본문에서 제외하고 metadata로 이동

- 청킹 장시간 실행 문제에 대해 worker 병렬화를 제안했고, 팀 기록상 실행 시간이 약 4시간에서 40분 수준으로 단축됐다.
- PDF 표 추출과 HWP/PDF 청크 샘플을 함께 확인하며 표의 Markdown·HTML 보존과 검색용 표현을 구분하도록 피드백했다.

5. Golden Set과 검색 성능 평가

- 512·1024·Semantic 세 청킹 조건을 같은 질문 집합으로 비교할 수 있도록 Golden Set 형식을 팀과 논의했다.
- 초기 문답이 모호하다는 피드백을 받아 질문과 정답 근거가 원문에서 명확히 확인되도록 생성 프롬프트와 양식을 다시 정리했다.
- Recall@K, Hit@K, MRR 평가 코드를 작성하고 청킹 조건과 검색 파라미터 조합을 비교했다.
- Reranker 실행 중 GPU OOM이 발생했을 때 batch size를 32에서 8로 낮춰 메모리 문제를 해결했다.
- 최종적으로 성능과 응답시간의 균형을 고려해 다음 구성을 선택했다.

항목	최종 선택
칭킹	1024 토큰 조건
BM25 / Vector 가중치	0.7 / 0.3
candidate_k	10, Reranker 후보 최대 20개
최종 반환	상위 10개
Recall@10	0.647
Hit@10	0.740
MRR	0.544
평균 처리시간	질문당 약 2.4초

candidate_k를 늘리면 Recall@10이 0.680으로 0.033 상승했지만 처리시간이 질문당 약 4.4초로 증가했다. 따라서 최고 점수만이 아니라 실제 서비스 응답시간까지 고려해 candidate_k=10을 최종안으로 판단했다.

6. UI·문서화·발표 기여

- Streamlit UI 구조를 설계하고 팀에 공유했다.
- UI와 RAG 코드 통합 과정에서 메뉴 연결, 근거 문서 표시, 검색 결과 개수 기준, 모델 설정과 Retriever 연결 방식을 검토했다.
- Naive RAG 및 검색 파트 내용을 문서화하고 README, 보고서, 발표 자료를 정리했다.
- LLM 질의 동작 영상을 녹화해 팀에 공유하고, 최종 GitHub 저장소 및 보고서 제출을 진행했다.
- 8월 3일 최종 발표 자료를 수정하고 팀 발표에 참여했다.

기간별 작업 흐름

기간	주요 작업	협업 결과
7/13~7/16	PM Kick-off, 역할 재배치, EDA, HWP 표 조사, 아키텍처 논의	협업 방식·업무 범위와 Naive/Advanced 분리 방향 공유
7/20~7/24	2주 일정·아키텍처 조율, Naive Retriever, Chroma 연동, 표·체크 검토	파트 간 산출물 연결과 전처리·칭킹 개선 피드백
7/27~7/30	세 칭킹 조건 검색, Golden Set, Reranker, Recall/MRR 평가	최종 검색 구성과 성능·속도 절충안 확정
7/31~8/3	UI·README·보고서·발표·제출	팀 산출물 정리와 최종 발표 완료

문제 해결에서 드러난 강점

- PM으로 변화와 의존성을 관리했다. 인원 변동 때 역할을 다시 배치하고, 각 파트의 완료 조건과 다음 담당자에게 필요한 산출물을 일정 안에서 연결했다.
- 검색 결과에서 데이터 문제를 역추적했다. 표가 적게 검색되는 현상을 단순히 임베딩 모델 문제로 보지 않고, 짧은 체크·오버랩·내부 ID 등 입력 단계까지 거슬러 확인했다.
- 성능 수치와 운영시간을 함께 비교했다. Recall 상승만 보고 후보 수를 늘리지 않고, 질문당 응답시간을 함께 비교해 실제 사용 가능한 최종안을 골랐다.
- 담당 영역 사이의 계약을 조율했다. 전처리·칭킹·인덱싱 결과가 Retriever와 Generation에서 어떻게 읽혀야 하는지 팀원들과 지속적으로 확인했다.
- 모호한 평가 기준을 보완했다. Golden Set의 질문·정답이 원문과 맞는지 다시 검토하고, 평가 데이터 자체의 품질도 검색 성능만큼 중요하게 다뤘다.